Kamiwaza 1.3.4 verification report

Evidence date: 2026-10-23

SANDBOX PROVIDERS: RELEASE VERIFIED · TESTED CONFIGURATION
70 / 87 capabilities accepted · +1 sandbox capability

Scope

Run a disposable sandbox worker, keep warm capacity ready, and return safely to the default provider. All 19 checks passed on the captured Core 1.3.4 configuration. This scoped result was accepted when capability-kit PR196 merged on October 23, 2026.

This operator/API recipe used Core 5f5f6465, the qualified ARM64 Operator and native controller, pinned upstream Agent Sandbox v0.5.2, and two retained worker image revisions. It did not test a graphical configuration screen. Exact source and image mappings are retained with the evidence.

Results

Accepted scoped release coverage is now 70 of 87 capabilities, up from 69. This adds one distinct capability: apps.sandbox-workload-provider. The 58 historical prerelease entries remain separate and unchanged. Nineteen passing checks are one capability, not nineteen additions.

Recipe: run disposable workers with ready capacity

Who: an operator authorized to configure an extension and its dedicated sandbox namespace, plus an application that calls the sandbox API.

Ingredients: a compatible Kamiwaza installation, the qualified sandbox controller and Operator, approved worker images, a worker-token Secret, configured network policies, and the pinned Agent Sandbox installation when choosing that provider. Use disposable resources and preserve their ownership identifiers.

1. Start with the default Kubernetes provider. From the authorized application, create a worker, list it, retrieve it, call its health endpoint and delete it. Confirm the worker disappears.

2. Opt the extension into Agent Sandbox. Configure the approved worker image, readiness endpoint, warm-pool size and maximum lifetime. Wait until the pool is ready before asking for a worker.

3. Create a worker from the ready pool. Retrieve its endpoint, confirm the authorized caller can reach it, and delete the claim when finished. Check that ready capacity is replenished.

4. Change between the two approved worker revisions. Check OnReplenish behavior, then Recreate behavior. Confirm requests for disallowed images or an image inconsistent with the pool are refused.

5. Renew a worker claim and check the renewed lifetime. Verify abandoned claims expire and the absolute maximum age still removes an actively renewed claim. Return to Kubernetes, create a worker with the requested worker-token environment entry, and verify it uses a Secret reference. Remove the disposable extension and confirm all owned resources are gone.

Expected result: both providers complete the worker lifecycle; access, image, expiry and token controls hold; switching providers preserves usable legacy workers. All 19 checks and cleanup passed. All 34 temporary fixture objects were confirmed absent after rollback.

END OF RECIPE

Ingredient glossary

Sandbox worker: a disposable standard container that performs application work. This proof does not establish a virtual-machine or stronger isolation boundary.

Kubernetes provider: the default Pod/Service worker backend.

Agent Sandbox provider: an optional backend using upstream claims and a pool of ready workers.

Warm pool: idle worker capacity prepared before an application requests it. OnReplenish and Recreate control how image changes replace that capacity.

Worker-token Secret: the credential presented to a worker. It is distinct from the controller token. Tested workers receive a Secret reference; ConfigMap token sources and foreign-owned mirror collisions are refused.

apps.sandbox-workload-provider: the single capability represented here. Status: release verified for the tested configuration.

Evidence

Accepted canonical source: https://github.com/kamiwaza-internal/capability-kit/tree/d16f17521f8d7ad9a824c99c7e903f0a35938bcb/reports/2026-10-23-sandbox-both-providers

Merged approval record: https://github.com/kamiwaza-internal/capability-kit/pull/196

Plan ID: a49bd5df0076fc2a4c1d371e360f848e034e8888172e327a04ea8fd4b569a165

The packet contains the release stamp, canonical record, stable before/after runtime observations, 101 structural transient samples, source qualification and ownership-bound cleanup journals. Independent review reproduced the stamp. Earlier failed attempts remain preserved rather than relabeled as passing.

Limits and assumptions

Evidence covers the captured ARM64 configuration and two worker revisions. It proves standard shared-kernel container behavior, not stronger isolation, hostile co-tenant containment, every installation configuration or human release approval. The operator/API recipe is not a verified end-user UI journey. Preserve these boundaries in customer reporting.

Provider rollback removes upstream claims, pool and template while retaining a usable worker-token mirror for legacy workers. The mirror is removed when its usable token configuration is removed; do not expect immediate mirror deletion merely from switching providers.

For an RFI, an assistant should report each requested requirement, the supported scope, its verification status, evidence and remaining gaps. Keep the tested scope and remaining gaps explicit; do not generate a bid response or infer accreditation, procurement advice or untested isolation claims.

Remaining work

Seventeen capabilities remain outside accepted release coverage. The accepted source snapshot and this report pair must be published together before website downloads reflect 70; this report pair alone does not change the canonical catalog. Full streaming proof remains a separate outstanding journey and would not add a new capability ID.